

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №5

з дисципліни
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44
Крошка Дар'я Олександрівна
номер у списку групи: 11

Перевірила:

Молчанова А. А.

Київ 2025

Постановка завдання:

Представити напрямлений граф із заданими параметрами так само, як у лабораторній роботі №3. Відмінність: коефіцієнт $k = 1.0 - 1 * 0.01 - 1 * 0.005 - 0.15$. Отже, матриця суміжності Adj матриці напрямленого графа за варіантом формується таким чином:

- 1) встановлюється параметр (seed) генератора випадкових чисел, рівне номеру варіанту $n_1 n_2 n_3 n_4$ ($n_1=4$ $n_2=4$ $n_3=1$ $n_4=1$);
- 2) матриця розміром $n * n$ заповнюється згенерованими випадковими числами в діапазоні $[0, 2.0)$;
- 3) обчислюється коефіцієнт $k = 1.0 - n_3 * 0.01 - n_4 * 0.005 - 0.15$, кожен елемент матриці множиться на коефіцієнт k ;
- 4) елементи матриці округлюються: 0 — якщо елемент менший за 1.0, 1 — якщо елемент більший або дорівнює 1.0.

2. Створити програму, яка виконує обхід напрямленого графа вшир (BFS) та вглиб (DFS).

- обхід починати з вершини із найменшим номером, яка має щонайменше одну вихідну дугу;
- при обході враховувати порядок нумерації;
- у програмі виконання обходу відображати покроково, черговий крок виконувати за натисканням кнопки у вікні або на клавіатурі.

3. Під час обходу графа побудувати дерево обходу. У програмі дерево обходу виводити покроково у процесі виконання обходу графа. Це можна виконати одним із двох способів:

- або виділяти іншим кольором ребра графа;
- або будувати дерево обходу поряд із графом.

4. Зміну статусів вершин у процесі обходу продемонструвати зміною кольорів вершин, графічними позначками тощо, або ж у процесі обходу виводити протокол обходу у графічне вікно або в консоль.

5. Якщо після обходу графа лишилися невідвідані вершини, продовжувати обхід з невідвіданої вершини з найменшим номером, яка має щонайменше одну вихідну дугу

Текст програми:

```
import numpy as np
import tkinter as tk
```

```
n1, n2, n3, n4 = 4, 4, 1, 1
```

```

n = 10 + n3
seed = int(f"{n1}{n2}{n3}{n4}")
k = 1.0 - n3 * 0.01 - n4 * 0.005 - 0.15

```

```

timer_delay = 500

```

```

np.random.seed(seed)
Adir = (np.random.rand(n, n) * 2.0 * k) >= 1.0
Adir = Adir.astype(int)
np.fill_diagonal(Adir, 0)

```

```

class GraphTraversalApp:

```

```

    def __init__(self, root, matrix):

```

```

        self.root = root
        self.matrix = matrix
        self.n = len(matrix)
        self.visited = set()
        self.tree_edges = []
        self.queue = []
        self.dfs_stack = []

```

```

        self.canvas = tk.Canvas(root, width=500, height=500, bg="white")
        self.canvas.pack()

```

```

        self.bfs_button = tk.Button(root, text="BFS Kpok", command=self.bfs_step)
        self.bfs_button.pack()
        self.dfs_button = tk.Button(root, text="DFS Kpok", command=self.dfs_step)
        self.dfs_button.pack()

```

```

        self.positions = {
            i + 1: (250 + 150 * np.cos(2 * np.pi * i / self.n), 250 + 150 * np.sin(2 * np.pi * i / self.n))
            for i in range(self.n)
        }
        self.draw_graph()

```

```

        self.matrix_label = tk.Label(root, text=self.format_matrix(), font=("Courier", 10))
        self.matrix_label.pack()

```

```

        candidates = [i + 1 for i in range(self.n) if any(self.matrix[i])]
        self.start_node = min(candidates) if candidates else None

```

```

        if self.start_node is not None:
            self.queue.append(self.start_node)
            self.dfs_stack.append(self.start_node)

```

```

    def format_matrix(self):
        return "\n".join(" ".join(map(str, row)) for row in self.matrix)

```

```

def draw_graph(self, highlight_edges=[], highlight_nodes={}):
    self.canvas.delete("all")

    for i in range(1, self.n + 1):
        for j in range(1, self.n + 1):
            if self.matrix[i-1, j-1] == 1:
                color = "red" if (i, j) in highlight_edges else "black"
                x1, y1 = self.positions[i]
                x2, y2 = self.positions[j]
                self.canvas.create_line(x1, y1, x2, y2, arrow=tk.LAST, fill=color, width=2)

    for i, (x, y) in self.positions.items():
        color = highlight_nodes.get(i, "lightblue")
        self.canvas.create_oval(x-15, y-15, x+15, y+15, fill=color, outline="black")
        self.canvas.create_text(x, y, text=str(i), font=("Arial", 12, "bold"))

def bfs_step(self):
    if not self.queue:
        unvisited = [i + 1 for i in range(self.n) if i + 1 not in self.visited and any(self.matrix[i])]
        if unvisited:
            self.queue.append(min(unvisited))
        return

    node = self.queue.pop(0)
    self.visited.add(node)

    for neighbor in range(1, self.n + 1):
        if self.matrix[node-1, neighbor-1] == 1 and neighbor not in self.visited:
            self.visited.add(neighbor)
            self.queue.append(neighbor)
            self.tree_edges.append((node, neighbor))
            self.draw_graph(highlight_edges=self.tree_edges, highlight_nodes={neighbor:
"green"})
            self.root.after(timer_delay, self.bfs_step)
    return

def dfs_step(self):
    if not self.dfs_stack:
        unvisited = [i + 1 for i in range(self.n) if i + 1 not in self.visited and any(self.matrix[i])]
        if unvisited:
            self.dfs_stack.append(min(unvisited))
        return

    node = self.dfs_stack.pop()
    if node in self.visited:
        return

    self.visited.add(node)

```

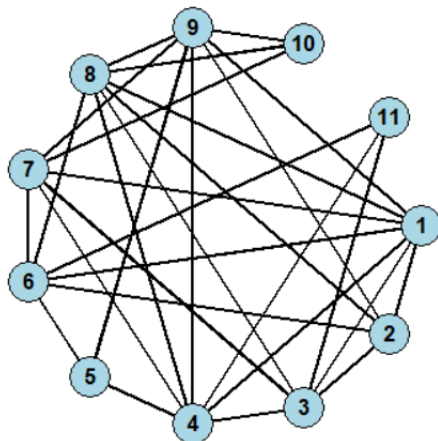
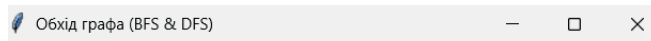
```
self.draw_graph(highlight_edges=self.tree_edges, highlight_nodes={node: "orange"})
```

```
for neighbor in range(self.n, 0, -1):
    if self.matrix[node-1, neighbor-1] == 1 and neighbor not in self.visited:
        self.tree_edges.append((node, neighbor))
        self.dfs_stack.append(neighbor)
        self.root.after(timer_delay, self.dfs_step)
    return
```

```
if __name__ == "__main__":
    root = tk.Tk()
    root.title("Обхід графа (BFS & DFS)")
    app = GraphTraversalApp(root, Adir)
    root.mainloop()
```

Тестування:

Вікно:

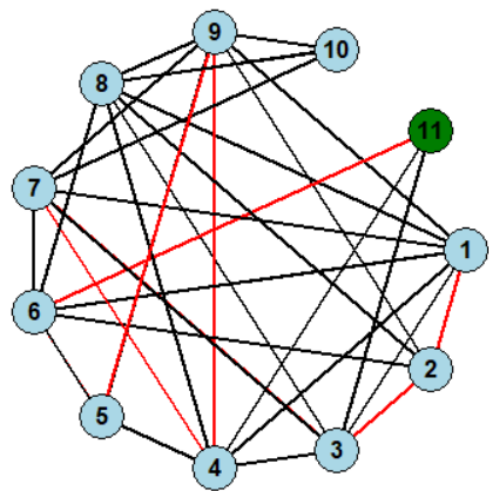
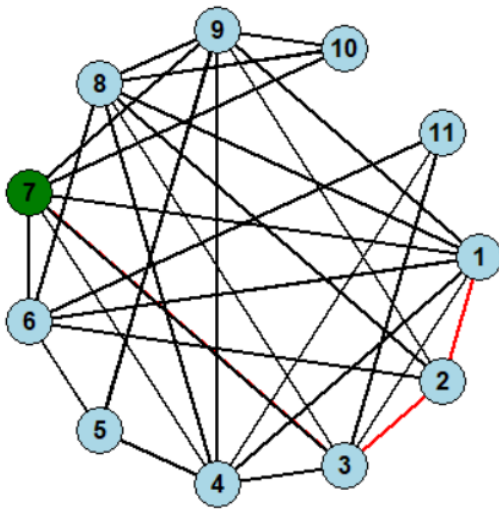


	BFS Крок										
	DFS Крок										
0	1	0	1	0	1	0	1	1	0	0	
0	0	1	0	0	0	0	0	1	0	0	
1	0	0	0	0	0	1	1	0	0	0	
0	0	1	0	0	0	0	0	1	0	0	
0	0	0	1	0	1	0	0	1	0	0	
1	1	0	0	1	0	1	0	0	0	1	
1	0	1	1	0	0	0	0	1	1	0	
0	1	0	1	0	1	0	0	0	1	0	
0	0	0	0	1	0	0	1	0	1	0	
0	0	0	0	0	0	0	1	0	0	0	
0	0	1	1	0	0	0	0	0	0	0	

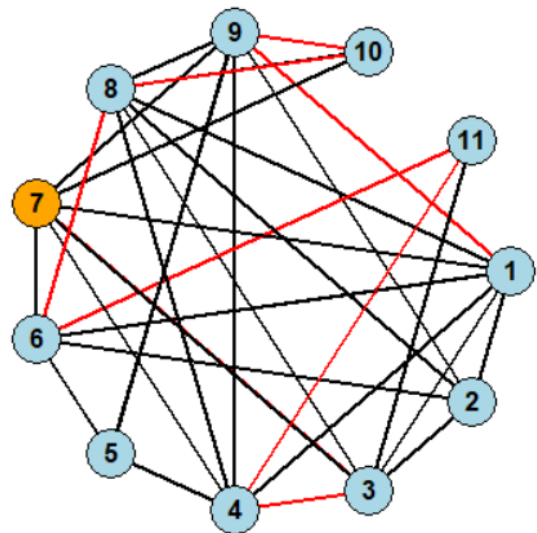
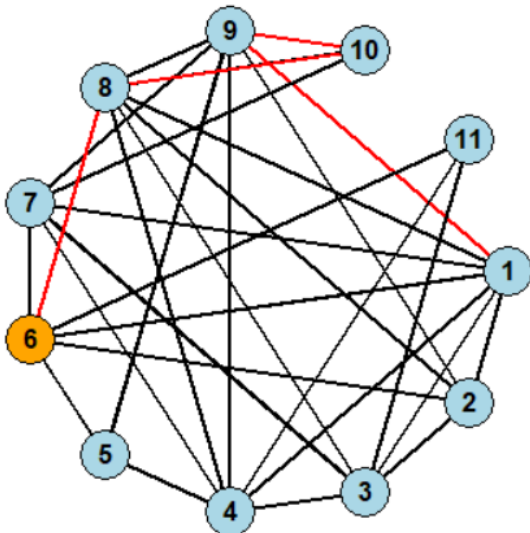
Матриця:

0	1	0	1	0	1	0	1	1	0	0
0	0	1	0	0	0	0	0	1	0	0
1	0	0	0	0	0	1	1	0	0	0
0	0	1	0	0	0	0	0	1	0	0
0	0	0	1	0	1	0	0	1	0	0
1	1	0	0	1	0	1	0	0	0	1
1	0	1	1	0	0	0	0	1	1	0
0	1	0	1	0	1	0	0	0	1	0
0	0	0	0	1	0	0	1	0	1	0
0	0	0	0	0	0	0	1	0	0	0
0	0	1	1	0	0	0	0	0	0	0

BFS:



DFS:



Розроблена програма успішно генерує та візуалізує напрямлений граф із 11 вершинами відповідно до заданих умов, використовуючи випадкову матрицю суміжності. Реалізовано покрокове виконання алгоритмів обходу в ширину (BFS) і глибину (DFS) з автоматичним вибором стартової вершини, зміною кольорів для відвіданих вершин та виділенням ребер дерева обходу. Інтерактивний графічний інтерфейс забезпечує зручне керування процесом та наочне представлення структури графа й результатів обходу.